VERIFICATION LEDGER

Every Headline Number, and How to Reproduce It

The research figures across all six projects, each with its source file and the command that regenerates it. Dated, because data moves.

Daniel Lichtenberger · October 2026
danny-397.github.io · Source on GitHub

Every headline number on this site, where it comes from, and the command that reproduces it. Figures are dated because data sources and package versions move; a rerun on a later date can legitimately differ, and the source files say how. Test counts are not listed here: they change with every commit, so the site reads them from each repository's CI instead.

Compiled 2026-10-28; RL-Trader rows updated 2026-11-14 (v2.3.0).

How RL agents fool you (RL-Trader)

Claim	Figure	Source	Reproduce
A single favourable run was not an edge	crypto agent **−40.5%** [−51.5%, −27.7%] across 30 seeds vs buy-and-hold +10.3%; behind on 5 of 6 coins, permutation **p = 0.19**	RESULTS.md §10h	`python tools/real_significance.py --seeds 30 --timesteps 150000`
The stock agent is reliably worse than holding	**−29.8%** [−35.0%, −24.3%] vs +239.5%, behind in all 300 runs, **p = 0.0021**	RESULTS.md §10h	same command
It holds across history	retrained in four periods: stocks behind in **4 of 4**, crypto in **3 of 4** (the fourth was a −46% crash it lost less in)	RESULTS.md §10h	`python tools/basket_walk_forward.py` (see RESULTS.md §12)
Why: no directional information	daily long-or-short calls right **50.0%** of the time (crypto; 49.8% stocks); daily turnover 0.52 of equity	RESULTS.md §10h	from the 30-seed command
The study survived an audit	pre-registered audit (2026-11-13): a repaired PPO matches Stable-Baselines3 on 3 of 4 benchmarks; a calendar leak in the crypto split fixed; no implementation, algorithm, budget or universe beats buy-and-hold	RESULTS.md §10	`.github/workflows/rigor.yml`
Memorisation, and the fix	on simulated paths, held-out return one fixed path → a fresh path every episode: equity-like **−25%** → **+31%**, crypto-like **−44%** → **+187%** (10 seeds, all four intervals exclude zero)	RESULTS.md §1	`python tools/ablation_multiseed.py --seeds 42 43 44 45 46 47 48 49 50 51 --timesteps 60000`
Pre-registered predictions	depth study (2026-11-14): **5 of 6 held**; failed: "never better in any period" (crypto, the 2025–26 crash)	docs/preregistration/2026-11-14-depth.results.md	`rigor.yml` job `depth`

QuantumSafe Scan

Claim	Figure	Source	Reproduce
Precision on the project's labeled corpus	**100%** precision and recall on 26 labeled findings (18 files, 9 languages); naive line-regex **49.1%**	benchmark/RESULTS.md	`python benchmark/evaluate.py`
Recall under mutation	**50/50** seeded cases found, 0 false positives on 50 negative mutations	benchmark/RESULTS-seeded.md	`python benchmark/seeded.py`
Precision on real code nobody chose	**93%** (95% CI 86–97%) on 100 random findings from 37 PyPI packages, each hand-labeled with a reason; measured before the TLS rule fix it exposed	benchmark/RESULTS-precision.md	`python benchmark/precision_sample.py --score`
Scale	**5,609** findings (4,236 HIGH) across **11,275** files in 37 packages, 2026-10-28, scanner 0.3.1	benchmark/RESULTS-realworld.md	`python benchmark/realworld.py`
Versus general security tools	quantum-relevant findings caught on the labeled corpus: QuantumSafe 26/26, Semgrep (`p/r2c-security-audit`) 4/26, Bandit 2/26	benchmark/COMPARISON.md	`python benchmark/comparison.py`

AlphaGlyph

Claim	Figure	Source	Reproduce
The best backtest in a search is probably overfit	probability of backtest overfitting **72%** over a 42-variant grid on SPY (CSCV); median out-of-sample Sharpe 0.65, ~52% of in-sample	technical report §7	`python tools/pbo_snapshot.py` (pinned SPY snapshot, SHA-256 recorded)
The ML forecaster cannot predict direction	test AUC **≈ 0.51**	technical report §7	model metadata, `ml_model_meta.json`
One conditional positive, and how fragile it is	Dip Buyer on AAPL/MSFT, 2021-01-01 → 2024-01-01: **+21.6%** vs SPY +5.6% in ranging markets; one more year shrinks the edge to **+3.6 points**	README, technical report §7	backtester with those dates; `regime_conditional` in the result

AlphaZero-style chess engine

Claim	Figure	Source	Reproduce
The plateau was a training-signal problem, not compute	Elo vs a random player: baseline **−21**, 10× self-play **−7**, material signal **+28**	docs/experiment-results.md	`python run_experiment.py --device cuda` (9.8 h on 2× T4)
Strength of the classical control engine	Easy (depth 2) ≈ **1390** [1284, 1512] on Stockfish's engine scale: +14 =8 −8 vs Stockfish 19 at `UCI_Elo 1320`	docs/data/stockfish_rating_d2_e1320.json	`python tools/stockfish_rating.py --stockfish <path> --depth 2 --games 30`

Tradeski and Neural Canvas

These two make engineering claims rather than research ones: Tradeski's indicators are hand-coded on NumPy and covered by its CI (Python and JavaScript suites); Neural Canvas runs MediaPipe's 21-landmark hand model on-device at roughly 60 frames a second. Both are live and can be checked directly.

Accessibility

All seven sites report **zero** violations under axe-core 4.10 with the WCAG 2.1 A/AA and best-practice rule sets (main pages and views, 2026-10-28).